Python Primer — read this BEFORE the tutorial

Who is this for? Anyone who has never (or barely) programmed. It covers only what you need for Tutorial 1, nothing more. Set aside about **1.5–2 hours**, and **type every example yourself** — reading is not the same as doing.

How to use it: finish the Setup Guide first. Then open a Jupyter notebook (or Google Colab), make a new notebook, and try each snippet in a cell. Press `Shift + Enter` to run a cell.

---

1. What is programming?

A program is a list of instructions the computer follows **one line at a time, top to bottom**. Python is a language that is easy to read, so the instructions look almost like English.

In Jupyter, the last line of a cell is shown automatically; to show anything else, use `print`.

```
print("Hello, world!")
```

Comments

Anything after `#` is ignored by Python. It is a note for humans.

```
# this line does nothing
print(5)      # this prints 5
```

---

2. Variables and basic types

A **variable** is a name attached to a value. Use `=` to create one.

```
mass = 2.5          # a decimal number (float)
n = 10              # a whole number (integer)
name = "pendulum"   # text (string)
done = True         # True or False (boolean)

print(mass, n, name, done)
```

Use `type(...)` to ask Python what something is:

```
print(type(mass), type(n), type(name))
```

> `=` means **“store this value in this name”**, not “equals” as in maths. `x = x + 1` is perfectly fine: take x, add 1, store it back in x.

Arithmetic

Operation	Symbol	Example	Result
Add, subtract	`+ -`	`7 - 2`	`5`
Multiply	`*`	`3 * 4`	`12`
Divide	`/`	`7 / 2`	`3.5`
Power	`**`	`2 ** 3`	`8`
Remainder	`%`	`7 % 2`	`1`

Careful: the power is `**`, not `^`. So x² is `x**2`.

```
g = 9.8
t = 2.0
distance = 0.5 * g * t**2
print(distance)          # 19.6
```

Strings

Text goes in quotes (single or double, your choice). You can print text and numbers together by separating them with commas:

```
print("The value of g is", 9.8, "m/s^2")
```

Exercise 2

Compute the kinetic energy $\frac{1}{2}mv^2$ for $m = 3$ and $v = 4$ and print it. (Answer: 24.0)

---

3. Lists

A **list** holds several values in order. It uses square brackets.

```
x = [1, 2, 3, 4, 5]
y = [1, 4, 9, 16, 25]

print(len(x))      # 5  (how many items)
```

Indexing — counting starts at ZERO

```
x = [10, 20, 30, 40, 50]
print(x[0])      # 10   (the first item)
print(x[2])      # 30
print(x[-1])     # 50   (the last item)
```

Slicing

`x[a:b]` gives items from position `a` up to **but not including** `b`.

```
print(x[1:3])    # [20, 30]
print(x[:2])     # [10, 20]   (from the start)
print(x[2:])     # [30, 40, 50]  (to the end)
```

Adding to a list

```
x.append(60)
print(x)
```

> A plain list cannot do maths on all its items at once. `[1, 2, 3] * 2` gives `[1, 2, 3, 1, 2, 3]`, which is probably not what you want! That is exactly the problem NumPy solves (Section 6).

---

4. Loops, conditions, functions (just the basics)

You will **not** need to write these in Tutorial 1 — but you will see them in the exercises and in other people's code, so recognise them.

`for` loop — repeat something for every item

Python uses **indentation** (4 spaces, or press Tab) to know what is inside the loop. This is mandatory.

```
for value in [1, 2, 3]:
    print(value * 10)
```

`range(5)` gives `0, 1, 2, 3, 4`:

```
for i in range(5):
    print(i)
```

`if` — make a decision

```
marks = 42
if marks >= 40:
    print("pass")
else:
    print("fail")
```

Comparisons: `==` (equal to), `!=` (not equal), `<`, `>`, `<=`, `>=`. Note `==` has two equals signs.

Functions

A function takes inputs and gives back an output. You have already used some: `print(...)`, `len(...)`.

Defining your own:

```
def square(a):
    return a * a

print(square(5))     # 25
```

Calling functions — arguments

Inputs can be given **by position** or **by name (keyword arguments)**. You will see names everywhere in Matplotlib:

```
def area(length, width):
    return length * width

print(area(3, 4))                  # by position
print(area(width=4, length=3))     # by name; the order doesn't matter now
```

Compare with the plotting call from the tutorial:

```
plt.plot(x, y, color="red", linestyle="--", label="sin(x)")
#             ^^^^ positional   ^^^^ keyword arguments (name = value)
```

---

5. Libraries and `import`

Python on its own is small. **Libraries** add extra abilities. To use one, `import` it. We give it a short nickname with `as` so we type less:

```
import numpy as np
import matplotlib.pyplot as plt
```

Afterwards, a function from the library is used as `nickname.function(...)`: `np.sin(...)`, `plt.plot(...)`. Think of `np.` as “from the NumPy toolbox, take…”.

> **Most common beginner error:** forgetting to run the import cell, giving `NameError: name 'np' is not defined`. Fix: run the cell with the imports (Jupyter cells must be run in order, top to bottom).

---

6. NumPy — arrays of numbers

NumPy gives us the **array**: like a list, but you can do maths on the whole thing at once, and it is very fast.

Making arrays

```
import numpy as np

a = np.array([1, 2, 3, 4])
print(a)
```

Maths on whole arrays ("vectorised")

No loops needed — the operation is applied to every element:

```
a = np.array([1, 2, 3, 4])

print(a * 2)          # [2 4 6 8]
print(a + 10)         # [11 12 13 14]
print(a ** 2)         # [ 1  4  9 16]
print(a + a)          # [2 4 6 8]
```

NumPy also has the common maths functions, which also work on whole arrays:

```
print(np.sin(a))
print(np.sqrt(a))
print(np.exp(-a))
print(np.pi)          # 3.14159...
```

> Angles are in **radians**, not degrees. `np.sin(np.pi / 2)` is 1.

`np.linspace` — evenly spaced numbers

`np.linspace(start, stop, n)` gives `n` numbers from `start` to `stop`, **including both ends**. This is how you make the x-axis for a plot of a function.

```
x = np.linspace(0, 10, 5)
print(x)              # [ 0.   2.5  5.   7.5 10. ]

x = np.linspace(0, 10, 100)
y = np.sin(x)         # sin of all 100 numbers at once
```

Useful statistics

```
data = np.array([4, 8, 6, 5, 7])

np.mean(data)        # average  -> 6.0
np.median(data)      # middle value -> 6.0
np.std(data)         # standard deviation (spread)
np.min(data)         # smallest
np.max(data)         # largest
len(data)            # how many items
```

2D arrays and `[:, 0]`

A table of numbers (rows and columns) is a **2D array**. Index it as `array[row, column]`.

```
table = np.array([[1, 10],
                  [2, 20],
                  [3, 30]])

print(table[0, 1])     # row 0, column 1  -> 10
print(table[:, 0])     # ":" means "all rows", column 0 -> [1 2 3]
print(table[:, 1])     # second column     -> [10 20 30]
print(table[1, :])     # row 1, all columns -> [ 2 20]
```

`table[:, 0]` is the pattern you will use to split a data file into separate columns.

Reading data from a file: `np.loadtxt`

If you have a file `mydata.csv` containing

```
# x,y
1,10
2,20
3,30
```

(CSV = “comma-separated values”; lines starting with `#` are ignored), then

```
data = np.loadtxt("mydata.csv", delimiter=",")
x = data[:, 0]
y = data[:, 1]
```

> **File paths:** `"data/pendulum.csv"` means “the file `pendulum.csv` inside the folder `data`, **relative to where the notebook is**”. If you get `FileNotFoundError`, the notebook is not in the right folder, or the file is not where you think. In Colab you must upload the file first.

Random numbers

```
rng = np.random.default_rng(1)   # the 1 is a "seed": same seed, same numbers
rng.random(5)                    # 5 random numbers between 0 and 1
```

`np.meshgrid` — a grid of points (used for 3D surfaces)

To draw a surface $z = f(x, y)$ you need z at **every combination** of x and y. `meshgrid` builds those combinations:

```
x = np.array([0, 1, 2])
y = np.array([0, 10])
X, Y = np.meshgrid(x, y)
print(X)       # [[0 1 2]
               #  [0 1 2]]
print(Y)       # [[ 0  0  0]
               #  [10 10 10]]
```

Now `X**2 + Y**2` works at every grid point simultaneously.

Fitting a line: `np.polyfit`

`m, c = np.polyfit(x, y, 1)` finds the best straight line $y = mx + c$ through your points. The `1` means a degree-1 polynomial (a straight line).

```
x = np.array([1, 2, 3, 4])
y = np.array([3.1, 4.9, 7.2, 8.8])
m, c = np.polyfit(x, y, 1)
print(m, c)     # slope ~ 2, intercept ~ 1
```

Getting two values out at once (`m, c = ...`) is called **unpacking**.

Exercise 6

1. Make `x = np.linspace(0, 2*np.pi, 50)` and compute `y = np.sin(x)**2`. Print `np.max(y)`. (Expected: close to 1.)
2. Create the 2D array above, and extract the second column.

---

7. Matplotlib — making plots

Matplotlib draws graphs. We always import its `pyplot` module as `plt`.

The basic recipe

1. Prepare x and y
2. Draw (`plt.plot` or `plt.scatter`)
3. Decorate (labels, title, legend, grid)
4. Show (`plt.show()`)

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.plot(x, y, label="sin(x)")
plt.xlabel("x")
plt.ylabel("y")
plt.title("A sine wave")
plt.legend()
plt.grid(True)
plt.show()
```

Which plot type?

Plot	Command	Use for
Line	`plt.plot(x, y)`	A function or a trend
Scatter	`plt.scatter(x, y)`	Individual measurements
Histogram	`plt.hist(data, bins=10)`	How values are distributed
Error bars	`plt.errorbar(x, y, yerr=0.1, fmt="o")`	Data with uncertainty
Contour	`plt.contourf(X, Y, Z)`	A 2D function seen from the top

Styling options (keyword arguments)

```
plt.plot(x, y, color="red", linestyle="--", marker="o", linewidth=2, label="my data")
```

Option	Examples
`color`	`"red"`, `"blue"`, `"green"`, `"black"`
`linestyle`	`"-"` solid, `"--"` dashed, `":"` dotted
`marker`	`"o"` circle, `"s"` square, `"^"` triangle

Saving

```
plt.savefig("myplot.png", dpi=300)    # BEFORE plt.show()
```

Several plots in one figure

`plt.subplots` returns a **figure** (the whole canvas) and **axes** (each small plot). Commands on axes get `set_` in front: `ax.set_xlabel(...)`, `ax.set_title(...)`.

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))   # 1 row, 2 columns
ax1.plot(x, np.sin(x))
ax2.plot(x, np.cos(x))
ax1.set_title("sin")
ax2.set_title("cos")
plt.show()
```

3D plots

```
fig = plt.figure()
ax = fig.add_subplot(projection="3d")
ax.plot(x, y, z)           # 3D line
ax.scatter(x, y, z)        # 3D points
ax.plot_surface(X, Y, Z)   # surface
```

Reading error messages

Python errors look scary but are very informative. **Read the last line first.**

Message	Usual cause
`NameError: name 'x' is not defined`	You never ran the cell that creates `x`, or misspelt it
`SyntaxError`	A missing bracket, quote, or colon
`IndentationError`	Inconsistent spaces at the start of a line
`FileNotFoundError`	Wrong file name or folder
`ValueError: x and y must have the same first dimension`	x and y have different lengths
`ModuleNotFoundError: No module named 'numpy'`	Library not installed — redo the Setup Guide

---

8. Using Jupyter notebooks

- A notebook is made of **cells**. A cell is either **code** or **markdown** (text).
- `Shift + Enter` — run the cell and move to the next one.
- Cells share memory: a variable made in one cell is available in later cells. But you must run the earlier cell first.
- Strange behaviour? Use `Kernel → Restart & Run All` for a clean start.
- Save often with `Ctrl + S`.

- Press `Esc` then `A` (insert cell above) or `B` (below); `D D` deletes a cell.

---

9. Self-check — can you do these?

Try to do all of these **without** looking at the notes above. If you get stuck, that's fine, bring your question to the tutorial.

1. Create a variable `r = 3` and compute the area of a circle, πr^2, using `np.pi`.
2. Make an array `t` of 50 numbers from 0 to 5, then compute `h = 20*t - 5*t**2`.
3. Plot `h` against `t` with axis labels and a title. (A ball thrown upwards!)
4. What is `np.max(h)`? When is it highest?
5. Save the plot as `ball.png`.

---

10. Resources

Python basics (pick ONE and follow it)

- Official Python Tutorial — https://docs.python.org/3/tutorial/ (sections 1–5 are enough)
- Python for Everybody (free course by Dr. Chuck) — https://www.py4e.com/
- Software Carpentry: *Programming with Python* — https://swcarpentry.github.io/python-novice-inflammation/
- Automate the Boring Stuff with Python (free to read online) — https://automatetheboringstuff.com/
- W3Schools Python tutorial (quick, interactive) — https://www.w3schools.com/python/

NumPy

- NumPy: the absolute basics for beginners — https://numpy.org/doc/stable/user/absolute_beginners.html
- NumPy quickstart — https://numpy.org/doc/stable/user/quickstart.html
- W3Schools NumPy tutorial — https://www.w3schools.com/python/numpy/

Matplotlib

- Matplotlib "Quick start guide" — https://matplotlib.org/stable/users/explain/quick_start.html
- Matplotlib tutorials — https://matplotlib.org/stable/tutorials/
- Matplotlib gallery (copy & modify examples) — https://matplotlib.org/stable/gallery/

Jupyter / Colab

- Jupyter Notebook documentation — https://jupyter-notebook.readthedocs.io/
- Google Colab (no installation needed) — https://colab.research.google.com/

Videos (YouTube)

- Search for *“Corey Schafer Python tutorial”* — clear, beginner-friendly
- Search for *“freeCodeCamp Python for Beginners”* — a full course in one video
- Search for *“Matplotlib tutorial for beginners”*

Getting unstuck

- Copy the **last line** of the error message into a search engine — someone has had the same error.
- The built-in help: in a cell, type `help(np.linspace)` or put the cursor inside the brackets and press `Shift + Tab`.

Tip: you do **not** have to memorise commands. Even experienced scientists look up how to make plots every time. What matters is understanding what the code is doing — and that comes from trying things.